

Water Tanker Distribution Optimization System

for Karachi, Pakistan

Design and Analysis of Algorithms (DAA)
Semester Project

Live URL: <https://water.33.jugaar.ai>

GitHub: github.com/tehzeeshahs4s-hub/water_tanker_system

Prepared by: Tehzeeb Shah

Date: September 2026

Table of Contents

1. Introduction & Problem Statement
2. Tools & Technologies
3. Software Requirements Specification (SRS)
4. System Architecture & Design
5. Data Structures & Graph Model
6. Dijkstra's Shortest Path
7. Greedy Resource Assignment
8. Dynamic Programming Scheduler
9. Backtracking Slot Allocator
10. Ford-Fulkerson Max Flow
11. Algorithm Complexity Comparison
12. API Design & Endpoints
13. Frontend & Role-Based Access
14. Deployment Architecture
15. Testing & QA
16. Troubleshooting Guide
17. Sample Walkthroughs
18. Future Enhancements
19. References

1. Introduction & Problem Statement

1.1 Project Overview

The Water Tanker Distribution Optimization System is a web-based application designed to optimize the delivery of water via tankers in Karachi, Pakistan. Karachi, with a population of over 16 million, faces severe water scarcity. Water is delivered by private tankers from treatment plants, dams, and wells to residential and commercial areas. This system uses 5 classical DAA algorithms to optimize routing, scheduling, resource assignment, conflict resolution, and flow distribution.

1.2 Problem Statement

Given a weighted graph $G(V, E)$ representing Karachi's road network, a set of water sources S , a set of demand areas A with water demands, a fleet of tankers T with varying capacities, and time slots for loading/unloading, we need to optimize:

- Shortest delivery routes from sources to areas (Dijkstra)
- Optimal tanker-to-area assignment (Greedy)
- Conflict-free delivery scheduling across time slots (DP)
- Hydrant slot allocation when multiple tankers compete (Backtracking)
- Maximum water flow distribution through the pipe network (Ford-Fulkerson)

1.3 Real-World Context

Karachi receives only about 550 MGD against a demand of over 1,200 MGD. Private tanker operators fill this gap, delivering water from treatment plants to neighborhoods. This creates a complex logistics problem involving route optimization, scheduling, and resource allocation.

1.4 Objectives

- Implement 5 DAA algorithms for water distribution optimization
- Build a production-ready web application with 4 role-based dashboards
- Provide real-time monitoring and analytics for all stakeholders
- Deploy with HTTPS, security headers, and production-grade infrastructure
- Achieve sub-100ms API response times for all endpoints

2. Tools & Technologies

2.1 Frontend Stack

Technology	Version	Purpose
React.js	18.x	UI framework with hooks & context
React Router	6.x	Client-side SPA routing
Chart.js	4.x	Data visualization
Leaflet.js	1.9.4	Interactive map for routes
Framer Motion	11.x	Animations and transitions
Tailwind CSS	3.x	Utility-first CSS
Font Awesome	6.5.1	Icon library

2.2 Backend Stack

Technology	Version	Purpose
Node.js	18+ LTS	JavaScript runtime
Express.js	4.x	REST API framework
In-Memory Store	N/A	Data storage (no DB)

2.3 DevOps

Tool	Version	Purpose
Caddy	2.x	Auto HTTPS (Let's Encrypt)
PM2	5.x	Process manager (auto-restart)
Nginx	1.x	Static file serving
Git/GitHub	2.x	Version control

2.4 Why These Choices

React for component-based architecture. Express.js for lightweight REST API. In-memory storage to focus on algorithms. Caddy for zero-config HTTPS. PM2 for auto-restart on failure.

3. Software Requirements Specification (SRS)

3.1 Functional Requirements

FR-01: User Authentication

Support 4 roles: Admin, Dispatcher, Driver, Customer. Each with dedicated dashboard. Demo credentials provided.

FR-02: Booking Management

Customers book tankers by area, size, date, time. Price calculated from distance factor and capacity.

FR-03: Dispatch Management

Dispatchers view pending bookings, assign tankers. Dispatch records: assigned, loading, en-route, delivered.

FR-04: Algorithm Execution

5 DAA algorithms via API + UI. Each returns results with step-by-step execution traces.

FR-05: Real-Time Dashboard

Real-time statistics with 30-second auto-refresh.

FR-06: Water Quality Monitoring

Track pH, TDS, turbidity, chlorine, bacteria. Scored against WHO standards.

3.2 Non-Functional Requirements

NFR-01: Performance

- API < 100ms
- Frontend load < 3s
- 100+ concurrent users

NFR-02: Security

- HTTPS via Caddy
- HSTS headers
- CORS configured

NFR-03: Usability

- Responsive design
- Dark/Light theme
- Role-based navigation

NFR-04: Reliability

- PM2 auto-restart
- Graceful error handling

3.3 Constraints

- No external database (in-memory)
- Single-server VPS
- Max 15 tankers, 18 areas

4. System Architecture & Design

4.1 High-Level Architecture

Three-tier: Presentation (React SPA), Application (Express.js API + algorithms), Data (in-memory). Caddy handles HTTPS and routing.

4.2 Request Flow

- 1. Browser -> https://water.33.jugaar.ai
- 2. Caddy serves React SPA
- 3. React Router handles client-side routing
- 4. Components fetch /api/* endpoints
- 5. Caddy proxies API to Express.js :7777
- 6. Express processes, runs algorithms, returns JSON
- 7. React renders response

4.3 Data Flow

```
Browser -> Caddy (:443) -> / (static) -> React build
                                -> /api/* -> Express (:7777) -> In-Memory Data
```

4.4 Role-Based Access

Role	Dashboard	Pages
Admin	5-tab overview	All 16 pages
Dispatcher	Dispatch Hub	Dispatch, Bookings, Drivers
Driver	My Dashboard	Deliveries, Route, Profile
Customer	My Dashboard	Book, Track, Payments

5. Data Structures & Graph Model

5.1 Road Network Graph

- Weighted undirected graph $G(V,E)$:
- V: 38 nodes (10 sources + 18 areas + 10 junctions)
 - E: ~80 weighted edges (distances in km)

5.2 Adjacency List

```
graph = {
  "WS-001": [{node:"J-001", weight:5.2}, {node:"J-002", weight:3.8}],
  "J-001": [{node:"WS-001", weight:5.2}, {node:"AREA-001", weight:3.1}]
}
```

5.3 Data Models

Tanker

```
{ tankerId:"TK-001", capacity:10000, status:"available", driverName:"Ahmed", fuelLevel:85 }
```

Booking

```
{ bookingId:"BK-001", customerName:"Fatima", areaId:"AREA-001", tankerSize:"large", price:9500, status:"scheduled" }
```

5.4 Matrix vs List

Aspect	Matrix	List	Choice
Space	$O(V^2)$	$O(V+E)$	List
Lookup	$O(1)$	$O(\text{degree})$	List
Best for	Dense	Sparse	List ($E \sim 2V$)

6. Dijkstra's Shortest Path

6.1 Purpose

Find shortest route from water source to delivery area in weighted road network.

6.2 Problem

Tanker at WS-001 needs shortest path to AREA-005 (DHA Phase V) considering varying distances.

6.3 Description

Maintains visited set and priority queue of unvisited nodes with tentative distances. Extracts minimum, relaxes neighbors. Terminates when destination reached.

6.4 Pseudocode

```
DIJKSTRA(G, src, dst):
    dist[src]=0, dist[others]=inf
    pq={src:0}, prev={}
    while pq not empty:
        u = pq.extractMin()
        if u==dst: break
        for (v,w) in G[u]:
            if dist[u]+w < dist[v]:
                dist[v]=dist[u]+w, prev[v]=u
    return path(src,dst)
```

6.5 Complexity

Aspect	Complexity	Explanation
Time	$O((V+E)\log V)$	Binary heap
Space	$O(V+E)$	Distances + adjacency

6.6 Why Not BFS

BFS is for unweighted graphs. Our network has varying weights. Dijkstra uses priority queue for non-negative weights.

6.7 Sample

```
WS-001 -> J-001 (5.2km) -> J-003 (4.1km) -> AREA-005 (6.8km)
Total: 16.1 km
```

7. Greedy Resource Assignment

7.1 Purpose

Assign tankers to areas minimizing total travel distance while satisfying demand.

7.2 Strategy

Assign closest available tanker to highest-demand area. Locally optimal, near-optimal globally.

7.3 Steps

- Sort areas by demand (descending)
- Sort tankers by capacity (descending)
- For each area, find closest available tanker (Dijkstra)
- Assign, mark unavailable; repeat

7.4 Complexity

Aspect	Complexity	Explanation
Time	$O(n*m*(V+E)\log V)$	n areas, m tankers, Dijkstra/pair
Space	$O(n+m)$	Assignments + available

7.5 Properties

Greedy choice property: locally optimal -> globally near-optimal.

Optimal substructure: remaining assignments independent of current.

7.6 Limitations

- May miss globally optimal solution
- For exact optimal: Hungarian Algorithm $O(n^3)$

8. Dynamic Programming Scheduler

8.1 Purpose

Schedule deliveries across time slots to minimize delay with hydrant constraints.

8.2 State

```
dp[i][t] = min delay for first i tankers using slots up to t
dp[i][t] = min(dp[i-1][s] + delay(tanker_i, slot_t))
delay = |preferred - assigned|
```

8.3 Constraints

- One tanker per hydrant per slot
- Valid time windows
- Variable loading duration
- Priority ordering

8.4 Complexity

Aspect	Complexity	Explanation
Time	$O(n \cdot T)$	n tankers, T=14 slots
Space	$O(n+T+H)$	Schedule + used slots

8.5 vs Greedy

Aspect	Greedy	DP
Time	$O(n \log n)$	$O(n \cdot T)$
Optimality	Local	Global
Constraints	Simple	Complex

9. Backtracking Slot Allocator

9.1 Purpose

Resolve conflicts when tankers compete for same hydrant slot.

9.2 Approach

Try all assignments recursively. On conflict, backtrack and try next slot. Exhaustive with pruning.

9.3 Pseudocode

```
BACKTRACK(idx, tankers, sources):
    if idx==n: return assignment
    for source in sources:
        for slot in 0..max:
            if available:
                ASSIGN(tanker,source,slot)
                if BACKTRACK(idx+1,...): return
            UNASSIGN // backtrack
```

9.4 Complexity

Aspect	Complexity	Explanation
Time	$O(m^n)$ worst	m sources x T slots
Space	$O(n+m*T)$	Recursion + tracking
Practical	$O(n*m*T)$	With pruning

9.5 vs DP

Aspect	DP	Backtracking
Approach	Bottom-up	Top-down + prune
Time	$O(n*T)$	$O(m^n)$ worst
Best for	Known constraints	Complex constraints

10. Ford-Fulkerson Max Flow

10.1 Purpose

Maximize water distribution through pipe network from sources to areas.

10.2 Network Model

```
WS-001 -> J-001 -> J-003 -> AREA-001
WS-001 -> J-002 -> J-004 -> AREA-002
WS-002 -> J-005 -> J-006 -> AREA-003
```

10.3 Concepts

- Capacity: max flow per pipe
- Flow: current water flow
- Residual: remaining capacity
- Augmenting path: path with capacity
- Bottleneck: min edge on path

10.4 Steps

- Create residual graph
- BFS find augmenting path
- Find bottleneck
- Add to total flow
- Update residuals
- Repeat until no path

10.5 Complexity

Aspect	Complexity	Explanation
Time	$O(E \cdot \text{max_flow})$	E edges, BFS per
Space	$O(V+E)$	Residual graph

10.6 Max-Flow Min-Cut

Max flow = min cut capacity. Bottleneck determines max deliverable water. Critical pipes = min-cut edges.

11. Algorithm Complexity Comparison

11.1 Comparison Table

#	Algorithm	Problem	Time	Space	Optimality
1	Dijkstra	Route	$O((V+E)\log V)$	$O(V+E)$	Optimal
2	Greedy	Assign	$O(n*m*Dijk)$	$O(n+m)$	Near-opt
3	DP	Schedule	$O(n*T)$	$O(n+T+H)$	Optimal
4	Backtrack	Slots	$O(m^n)$	$O(n+m*T)$	Optimal
5	Ford-F	Flow	$O(E*\max F)$	$O(V+E)$	Optimal

11.2 When to Use

Scenario	Algorithm	Reason
Route A->B	Dijkstra	Shortest path
Assign tankers	Greedy	Fast, near-optimal
Schedule slots	DP	Constraints, optimal
Resolve conflicts	Backtrack	Exhaustive
Max flow	Ford-F	Bottleneck

11.3 Scaling

Size	Dijkstra	Greedy	DP	Backtrack	FF
n=10	<1ms	<1ms	<1ms	<1ms	<1ms
n=50	<1ms	2ms	<1ms	5ms	3ms
n=100	2ms	15ms	<1ms	timeout	10ms
n=500	10ms	200ms	2ms	timeout	50ms

12. API Design & Endpoints

12.1 Overview

35+ REST endpoints. All return JSON. RESTful conventions: GET read, POST create, PUT update, DELETE remove.

12.2 Core Endpoints

Method	Endpoint	Description
GET	/api/bookings	List bookings
POST	/api/bookings	Create booking
PUT	/api/bookings/:id	Update booking
DELETE	/api/bookings/:id	Delete booking
GET	/api/fleet	List tankers
GET	/api/fleet/stats	Fleet stats
GET	/api/drivers	List drivers
GET	/api/quality	Quality tests
GET	/api/payments	Payments
GET	/api/dispatches	Dispatches

12.3 Algorithm Endpoints

Method	Endpoint	Description
POST	/api/algorithms/dijkstra	Dijkstra
POST	/api/algorithms/greedy	Greedy
POST	/api/algorithms/dp-schedule	DP
POST	/api/algorithms/backtrack	Backtrack
POST	/api/algorithms/maxflow	Ford-F
POST	/api/algorithms/bfs	BFS
POST	/api/algorithms/priority-based	Priority
POST	/api/algorithms/fcfs-slot	FCFS
POST	/api/analytics/benchmark	Benchmark

12.4 Sample

```
POST /api/algorithms/dijkstra
Req: {"source": "WS-001", "destination": "AREA-005"}
Res: {"success": true, "path": ["WS-001", "J-001", "J-003", "AREA-005"], "distance": 16.1}
```

13. Frontend & Role-Based Access

13.1 Stack

React 18 + hooks. Context API (Auth, Theme). React Router v6. Tailwind CSS. Chart.js. Leaflet.js.

13.2 4 Dashboards

Role	Dashboard	Features
Admin	5-tab overview	Overview, Bookings, Fleet, Quality, Finance
Dispatcher	Dispatch Hub	Pending bookings, active dispatches, alerts
Driver	My Dashboard	Profile, deliveries, progress bars
Customer	My Dashboard	Book, track, payments, quality

13.3 Auth Flow

1. Click demo card -> 2. AuthContext.login() -> 3. Store localStorage -> 4. Redirect /dashboard -> 5. RoleSidebar renders

13.4 Theme

Dark/Light via ThemeContext. Persisted in localStorage. All components use useTheme() hook.

13.5 Features

- Dark/Light toggle
- Responsive
- Auto-refresh 30s
- Chart.js
- Leaflet maps
- Collapsible sidebar

14. Deployment Architecture

14.1 Stack

```
Internet -> Caddy (:443) -> / (static) -> React build
        -> /api/* -> Express (:7777) -> In-Memory
```

14.2 Steps

- Build: `cd client && npm run build`
- Deploy: `cp -r build/* /var/www/water-tanker/`
- Start: `pm2 start standalone-server.js --name water-tanker`
- Configure Caddy
- Reload: `systemctl reload caddy`

14.3 Caddy Config

```
water.33.jugaar.ai {
  @api path /api/*
  handle @api { reverse_proxy localhost:7777 }
  handle { root * /var/www/water-tanker
    try_files {path} /index.html
    file_server }
}
```

14.4 Security

Header	Value
HSTS	max-age=31536000
X-Content-Type	nosniff
Referrer-Policy	strict-origin-when-cross-origin
CORS	*

15. Testing & QA

15.1 Results

Check	Result	Details
API (12)	12/12 PASS	200 + JSON
Algorithms (13)	13/13 PASS	200
Pages (19)	19/19 PASS	200
Data (6)	6/6 PASS	Fields OK
Methods (4)	4/4 PASS	CRUD
Security	PASS	HSTS, nosniff
HTTPS	PASS	Caddy auto-SSL
Response	PASS	55-74ms

15.2 19 SQA Fixes

- BookingPage: area->areald, deliveryDate->date
- BookingsListPage: 4 field mismatches
- QualityPage: score fallback, bacteria check
- PaymentsPage: lowercase method keys
- DriverPage: licenseNo, status, form JSON
- MaintenancePage: type/date fields
- ResourceAssignment: averageDistance->totalDistance
- Server: Set->Array serialization

15.3 Performance

Metric	Target	Actual
API Time	<100ms	55-74ms
Load	<3s	~1.5s
Bundle	<1MB	802KB

16. Troubleshooting Guide

16.1 Issues & Solutions

Blank page

Cache issue. Fix: Ctrl+Shift+R hard refresh.

API 404

Server down. Fix: pm2 list, pm2 restart water-tanker

HTTPS error

DNS/Caddy issue. Fix: dig +short water.33.jugaar.ai, systemctl reload caddy

Empty dispatch

Field mismatch. Fix: verify bookingId, tankerId, driverId in API response

Theme broken

Context error. Fix: ensure useTheme() returns {colors, themeName, toggleTheme}

Slot grid empty

Set serialization. Fix: Array.from(set) before JSON response

Form submit fail

FormData vs JSON. Fix: JSON.stringify() + Content-Type: application/json

16.2 Debug Commands

```
pm2 list
pm2 logs water-tanker
curl localhost:7777/api/bookings
systemctl status caddy
caddy validate --config /etc/caddy/Caddyfile
dig +short water.33.jugaar.ai
```

17. Sample Walkthroughs

17.1 Customer Books Water

- Login as customer@aqua.com
- Click Book Water Now
- Fill: Name, Phone, DHA, Large, Date, Time
- Price: PKR 7000 x 1.4 = PKR 9800
- POST /api/bookings -> BK-016
- SMS + WhatsApp notification sent

17.2 Dispatcher Assigns

- Login as dispatcher@aqua.com
- See 6 pending, 9 available tankers
- Select BK-016 (DHA, Large)
- Greedy suggests TK-003 (closest)
- POST /api/dispatches -> DSP-021

17.3 Route Optimization

- Admin -> Route Optimizer
- Select WS-001 -> AREA-005
- POST /api/algorithms/dijkstra
- Result: 16.1 km path
- Leaflet map renders route

17.4 Slot Conflict

- Admin -> Slot Allocator
- 8 tanker requests to backtracking
- Tries all slot assignments
- Backtracks on conflicts
- Returns optimal schedule

18. Future Enhancements

18.1 Short-Term

- MongoDB for persistence
- WebSocket for real-time
- SMS/WhatsApp via Twilio
- Email verification

18.2 Medium-Term

- React Native mobile app
- GPS tracking
- ML demand prediction
- Reinforcement learning scheduling

18.3 Long-Term

- Multi-city (Lahore, Islamabad)
- IoT water sensors
- Blockchain payments
- Government integration

19. References

- Cormen et al. "Introduction to Algorithms" (CLRS), 4th Ed, 2022
- Dijkstra "A note on two problems in connexion with graphs" (1959)
- Ford & Fulkerson "Flows in Networks" (1962)
- Tarjan "Data Structures and Network Algorithms" (1983)
- React: <https://react.dev>
- Express.js: <https://expressjs.com>
- Caddy: <https://caddyserver.com/docs>
- KWSB Reports
- WHO Drinking Water Quality Guidelines, 4th Ed